

第5章 内存管理

建议学时:4-6 小时 | 难度:★★★☆☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 理解 Python 的"一切皆堆对象":没有 C 的栈上变量,所有对象都在堆上由引用计数管理
- 掌握**引用计数**的工作原理:绑定/解绑/函数传参/容器操作如何改变计数
- 掌握**分代 GC**:什么时候触发、能做什么(循环引用)、不能做什么(及时释放)
- 掌握循环引用与 `__del__` 的经典坑,学会用 `weakref` 打破循环
- 掌握 `with` 上下文管理器:内置用法与自定义 `__enter__` / `__exit__`、`contextlib`
- 理解"GC 语言里内存泄漏怎么发生":全局容器、缓存、闭包、循环引用
- 会用 `tracemalloc` 定位内存增长,养成资源释放的生产纪律

💡 从 C 到 Python:本章主题如何迁移

C 的内存模型是"栈 + 堆 + 静态区",程序员用 `malloc` / `free` 手动管理堆,用 `calloc` / `realloc` 调整,还要小心悬垂指针、重复释放、内存泄漏。Python 把这一切拿走:解释器为每个对象在**堆上**分配内存,用**引用计数**跟踪"还有谁在用它",计数归零立即回收(`__del__` 与内存释放都在此刻发生);堆内存不足时,分代垃圾回收器清扫"互相引用但没有外部引用"的对象环。这就是 CPython 的"引用计数 + 分代 GC"双轨制。

这套设计的直接后果:**你几乎不会"泄漏"内存,也几乎不能"控制"释放时机**。C 里 `free(ptr)` 是精确的;Python 里"什么时候释放"取决于引用什么时候消失——可能立即(计数归零),可能延迟(在 GC 环里),可能永久(还被全局引用)。所以生产调优的姿势完全不同:C 是"找漏 free",Python 是"确保引用图不膨胀"。

另一个大迁移是**资源释放**。C 里文件句柄、锁、网络连接靠手动 `fclose` / `unlock` / `close`,漏一次就泄漏一次。Python 的 `with` 语句是 RAII 的 Python 版:进入时 `__enter__`,退出时(无论正常、`return`、还是异常) `__exit__` 必然执行。C++ 程序员会心一笑,这是 RAII 的思想;C 程序员需要记住:打开资源的地方,立刻用 `with` 包起来,没有例外。

最后,GC 语言也有泄漏——只是形态不同:全局容器只增不减、缓存永不淘汰、闭包意外捕获大对象、回调列表不断累积。本章实验会复现并修复一个真实的"泄漏"。记住结论:在 Python 里排查内存问题,先画引用图,再谈优化。

📦 前置知识自查(你已会的 C 基础)

- `malloc` / `calloc` / `realloc` / `free` 的契约与内存泄漏、悬垂指针、双重释放
- 栈帧与局部变量的生命周期;函数返回后栈上变量失效
- `fopen` / `fclose` 等资源句柄的手动管理
- 静态/全局变量的生命周期(程序全程存活)
- 指针别名:两个指针指向同一块内存

🚀 本章学习路线

1. **第一阶段(1.5 小时,引用计数)**:读 §5.1–§5.2,用 `sys.getrefcount` 亲手验证计数变化
2. **第二阶段(1 小时,GC)**:读 §5.3–§5.4,复现循环引用与 `__del__` 坑,用 `weakref` 修复
3. **第三阶段(1.5 小时,资源管理)**:读 §5.5,写自定义上下文管理器
4. **第四阶段(实验,1.5 小时)**:完成章末实验"泄漏复现与修复"
5. **验收标准**:能解释"为什么 `del x` 之后内存不一定立刻归还"与"弱引用是什么"

本章知识导图

- §5.1 内存模型:栈/堆/静态区 vs 一切皆堆对象
- §5.2 引用计数:`getrefcount`、绑定/解绑、`del`
- §5.3 分代 GC:循环引用、`gc` 模块、阈值
- §5.4 `__del__` 的坑与 `weakref` 弱引用
- §5.5 `with` 上下文管理器与 `contextlib`
- §5.6 泄漏的四种形态与 `tracemalloc` 排查
- §5.7 生产实践:拼接性能、对象复用、内存纪律

§5.1 内存模型:一切皆堆对象

C 里 `int x = 5`; 把 5 放在栈上; `int *p = malloc(...)` 在堆上。Python 里**所有对象都在堆上**——包括整数、字符串这些"小东西"。名字(变量)只是指向堆对象的引用,它们存在于模块/函数的命名空间中。下表是内存模型对照:

C	Python
栈上局部变量,函数返回自动失效	名字绑定到堆对象;作用域结束,名字消失,引用计数减一
<code>malloc / free</code> 手动管理堆	引用计数归零立即回收;循环引用由 GC 回收
静态/全局变量全程存活	模块级名字全程存活——它们指向的对象"永不释放"
指针别名需小心	引用即别名,处处都是(第1章 <code>b = a</code>)
未初始化内存是垃圾值	没有未初始化对象;对象创建即完整初始化

★ 重点:名字与对象分离

名字 = 引用,对象 = 堆上的实体。内存的"生死"由对象的引用计数决定,不由名字决定。`del x` 只是删除名字;对象是否被回收,取决于还有没有别的名字/容器引用它。

§5.2 引用计数

CPython 对每个对象维护一个计数:谁引用它,计数加一;引用消失,计数减一;归零,立即释放内存。哪些操作会改变计数?

- 赋值 `y = x`、传入函数、放进 `list/dict`、作为返回值——计数 +1
- `del x`、函数返回、从容器删除、作用域结束——计数 -1
- 注意: `sys.getrefcount(x)` 本身会临时 +1,所以结果比直觉多 1

这与 C 的堆管理形成最鲜明的对照——C 靠人,Python 靠机制:

C(malloc/free)	Python(引用计数)
申请/释放时机由程序员决定	由"最后一个引用消失"自动决定
漏 free 或重复 free 都是 bug	没有"重复释放";漏释放表现为引用还活着
悬垂指针:free 后仍持有地址	无悬垂指针:对象没了名字/引用也不存在
free 立即归还,时机精确	计数归零立即归还;循环引用等 GC,时机不定
崩溃通常发生在"错误的地点和时间"	错误通常表现为"内存曲线缓慢上升"

示例 5-1 观察引用计数

```
import sys

x = [1, 2, 3]          # 对象创建,1 个引用(x)
print(sys.getrefcount(x)) # 2 = 真实 1 + getrefcount 自己临时 +1

y = x                  # y 也引用 → 计数 2
print(sys.getrefcount(x)) # 3

lst = [x]               # 列表持有引用 → 3
print(sys.getrefcount(x)) # 4

del y                  # 少一个引用 → 2
print(sys.getrefcount(x)) # 3

lst.clear()            # 列表不再引用 → 1
print(sys.getrefcount(x)) # 2

del x                  # 最后一个引用消失 → 对象立即释放
# print(sys.getrefcount(x)) # NameError: 名字都不存在了

# 小整数和短字符串有缓存,别拿它们测计数(会误导)
```

⚠ 误区 1:以为 del 立即释放内存

del x 只删名字。对象还活着,直到**最后一个引用**消失(或被 GC 从循环中回收)。函数里的局部变量在函数返回时统一减引用;闭包捕获的变量活得比函数长(第4章);全局变量活到进程结束。判断"何时释放"的唯一方法:数引用——这也是 weakref (§5.4)存在的原因。

§5.3 分代 GC:专治循环引用

引用计数解决不了**循环引用**:A 引用 B,B 引用 A,但外面没人引用它们——计数永远 ≥ 1 ,永不归零。分代 GC(标记-清除 + 分代)定期扫描,找出这种"孤岛"并回收。要点:

- **什么时候触发**:分配次数超过阈值时(默认 700/10/10,三代); gc.collect() 手动触发
- **能做什么**:回收循环引用;unreachable 对象若定义了 `__del__`,会被移入 `gc.garbage`
- **不能做什么**:不能按你的时间表回收——它不是引用计数那种"立即"语义
- 只有**容器**(list/dict/set/对象实例)参与 GC;int/str 不参与(它们造不出循环)

示例 5-2 循环引用与手动回收

```
import gc

class Node:
    def __init__(self, name):
        self.name = name
        self.other = None

def make_cycle() -> None:
    a = Node("A")
    b = Node("B")
    a.other = b          # A → B
    b.other = a          # B → A:形成环,函数返回后无人引用

gc.disable()            # 先关掉自动 GC, 观察
make_cycle()
print("回收前:", len(gc.get_objects()))    # 环还在
n = gc.collect()        # 手动回收
print(f"回收了 {n} 个对象")    # 环被标记-清除回收
print("回收后:", len(gc.get_objects()))
gc.enable()             # 生产代码千万别留 gc.disable()

# 生产注意:gc.get_objects() 开销大,只用于调试
```

⚠ 误区 2:依赖 GC 来"及时"释放资源

文件句柄、锁、连接这类资源**不能等 GC**:循环引用里的对象可能要等很多轮 GC 才被清扫,句柄就泄漏了。而且 `__del__` 方法在 GC 回收时的调用时机不可预测。生产铁律:资源释放永远用 `with / try-finally` 显式控制 (§5.5),GC 只负责内存,不负责资源。另一个极端:不要没事调 `gc.collect()`,它是全停顿操作,在服务里会引发卡顿。

§5.4 `__del__` 的坑与 `weakref` 弱引用

`__del__` 是"对象被回收前"的钩子,但有三宗罪:①时机不可预测(引用计数归零时立即,但循环引用里要等 GC);②循环引用里它可能导致对象永远留在 `gc.garbage`;③解释器退出时全局变量不保证调用。C 程序员会把它当析构函数用——**不要**。析构逻辑的正确位置是上下文管理器 (§5.5)或显式 `close()` 方法。

弱引用(`weakref.ref`)不增加引用计数:目标存活时,弱引用可访问;目标被回收,弱引用返回 `None`。它专治两件事:①打破循环引用;②缓存(C 的"缓存但允许淘汰")。

示例 5-3 `__del__` 的坑与 `weakref`

```

import gc
import weakref

class Resource:
    def __init__(self, name):
        self.name = name
        self.peer = None

    def __del__(self):          # 坑:依赖它的时机就是错的
        print(f"{self.name} 被回收")

r1, r2 = Resource("r1"), Resource("r2")
r1.peer, r2.peer = r2, r1      # 循环引用
del r1, r2                    # 两个对象计数都 >= 1
print("--- 手动 GC ---")
gc.collect()                  # 这里才回收;若 __del__ 有副作用,顺序不定

# weakref:不增加计数,目标死了就是 None
class Cache:
    pass

obj = Cache()
ref = weakref.ref(obj)        # 弱引用
print(ref() is obj)           # True:目标活着
del obj
print(ref())                   # None:目标已回收

# 弱引用打破循环:one 只弱引用 two
class Partner:
    pass

one, two = Partner(), Partner()
one.other = weakref.ref(two)   # 弱引用 → 不构成环
two.other = one
del two
print(one.other())             # None:two 已被回收(计数归零)

```

工程建议:缓存的正确姿势

全局缓存 (CACHE = {}) 会让所有被缓存对象永久存活——这本身是一种“泄漏”。两个对策：① `weakref.WeakValueDictionary`：对象被别处释放后自动从缓存消失，缓存永不膨胀；② `functools.lru_cache` (第12章)自带大小上限。生产里，“缓存必须有限”是内存纪律的第一条。

§5.5 with 上下文管理器

`with` 语句 = C++ RAII 的 Python 版：**进入执行 `__enter__`，退出(正常/异常/return)必然执行 `__exit__`**。文件、锁、连接、临时目录……凡是有“用完必须关”语义的对象，都用 `with`。它等价于：

```
# 下面的 with 语句:
with open("f.txt", encoding="utf-8") as fh:
    data = fh.read()

# 等价于 try/finally(别自己写,with 更清晰):
fh = open("f.txt", encoding="utf-8")
try:
    data = fh.read()
finally:
    fh.close()
```

资源释放的 C/Python 对照——C 靠人肉记住,Python 靠语法保证:

C(手动释放)	Python(with 上下文)
<code>FILE *fp = fopen(...); ... fclose(fp);</code>	<code>with open(...) as fp: ...</code>
pthread_mutex_lock/unlock 成对	<code>with lock:</code> 解锁必然执行
中间 return/异常 → 漏释放	return/异常 → <code>__exit__</code> 依然执行
资源计数全靠审查纪律	资源计数可由代码评审强制("一律 with")
close 后再次使用是 UB	退出 with 后访问资源对象同样报错(closed)

5.5.1 自定义上下文管理器

自己写资源类时,实现 `__enter__` / `__exit__`;只想包一层逻辑(计时、加锁)时,用 `@contextmanager` 装饰器把生成器函数变成上下文管理器——`yield` 前后分别是进入/退出逻辑。

示例 5-4 自定义上下文管理器

```
# 方式一:类实现 __enter__/___exit__
class Timer:
    """测量 with 块执行耗时"""

    def __enter__(self):
        import time
        self.start = time.perf_counter()
        return self          # as 子句拿到它

    def __exit__(self, exc_type, exc_val, exc_tb):
        import time
        cost = time.perf_counter() - self.start
        print(f"耗时 {cost * 1000:.2f} ms")
        return False         # False:异常继续向外抛

with Timer():
    total = sum(range(1_000_000))

# 方式二:@contextmanager 生成器版
from contextlib import contextmanager

@contextmanager
def timed(name):
    import time
    start = time.perf_counter()
    try:
        yield                # with 块体在这里执行
    finally:
        cost = time.perf_counter() - start
        print(f"[{name}] 耗时 {cost * 1000:.2f} ms")

with timed("排序"):
    data = list(range(1000, 0, -1))
    data.sort()

# 内置常用:contextlib.suppress / closing / ExitStack
from contextlib import suppress, closing, ExitStack
```

⚠ 误区 3:忘记 with 或手动 open/close

`fh = open(...); ...; fh.close()` 是 C 时代思维——中间任何一行抛异常,close 就不执行,句柄泄漏。规矩: **open 的一瞬间就用 with 包起来**;多资源用 `with A() as a, B() as b:` 或 `ExitStack`。同理,锁、连接、临时文件全部用 with。代码评审看到"裸 open 后 5 行再 close",直接打回。

§5.6 泄漏的四种形态与 tracemalloc

GC 语言也会"泄漏",但形态不同——本质都是"引用图只增不减"。四种典型:

- **全局容器累积**:日志列表、请求记录 append 后从不清理

- **缓存无上限**:字典缓存键不断增加
- **闭包捕获**:回调闭包意外捕获大对象,生命周期被拉长
- **事件/回调注册未注销**:观察者列表只加不减

tracemalloc 是标准库内存追踪器,可快照对比定位增长来源:

示例 5-5 tracemalloc 定位内存增长

```
import tracemalloc

tracemalloc.start()          # 开始追踪(生产调试时才开)

# 模拟泄漏:全局列表只增不减
leaked = []
for i in range(1000):
    leaked.append({"id": i, "payload": "x" * 1000})

snapshot = tracemalloc.take_snapshot()
top = snapshot.statistics("lineno")    # 按代码行统计
for stat in top[:3]:
    print(stat)                  # 显示"哪一行分配了多少"

# 输出示例(行号视实际文件):
# /path/demo.py:10: size=1000 KiB, count=1000, average=1.0 KiB
# → 定位到第 10 行,就是泄漏源头

tracemalloc.stop()

# 对比两次快照更实用:
# s1 = take_snapshot(); 运行可疑代码; s2 = take_snapshot()
# s2.compare_to(s1, 'lineno') 列出增长最大的行
```

§5.7 生产实践:性能与纪律

三个高频内存/性能话题:字符串拼接、对象复用、内存视图。

示例 5-6 拼接性能与对象复用


```
# 1) 字符串拼接:+= 是  $O(n^2)$ ,join 是  $O(n)$ 
parts = [f"行{i}" for i in range(10000)]
text = ""
for p in parts:
    # 反例:每次 + 都新建字符串并拷贝
    text += p + "\n"

text = "\n".join(parts)    # 正例:一次性拼接(生产标准)

# 2) 大量小对象复用:数据类/元组优先于 dict
import timeit
t_dict = timeit.timeit(
    '{"x": 1, "y": 2}', number=100000)
t_tup = timeit.timeit(
    '(1, 2)', number=100000)
print(f"dict 构造 {t_dict:.3f}s vs 元组 {t_tup:.3f}s")
# 元组显著更快;百万级循环里,打包用元组

# 3) 大文件不整读:逐行/分块处理,避免一次性内存峰值
# (第7章详述;这里记住:能流式就不要全量)
```



工程建议:内存纪律清单

① 资源用 with,不用裸 open;② 缓存必须有限(WeakValueDictionary/lru_cache 上限);③ 全局容器谨慎 append——考虑 deque(maxlen=) 或定期清理;④ 大数据用流式而非全量;⑤ 上线前用 tracemalloc/psutil 做一次"长跑内存"压测,观察内存曲线是否收敛。C 程序员要改的直觉:不再问"这块内存谁释放",改问"这个对象的引用图会不会无限膨胀"。

本章速查表

主题	结论 / 写法
内存模型	一切皆堆对象;名字只是引用
释放时机	引用计数归零立即;循环引用等 GC;全局引用永不
查看计数	<code>sys.getrefcount(x)</code> (自带 +1)
删除名字	<code>del x</code> 只减引用,不保证释放
循环引用	list/dict/对象环; <code>gc.collect()</code> 手动回收
gc 阈值	默认 700/10/10;别随意 disable
<code>__del__</code>	时机不可预测;析构逻辑放 with/close()
弱引用	<code>weakref.ref(obj)</code> 不增计数;目标死了返回 None
弱缓存	<code>weakref.WeakValueDictionary</code> 自动淘汰
with	进入 <code>__enter__</code> ,退出必然 <code>__exit__</code>
自定义上下文	类实现 <code>__enter__</code> / <code>__exit__</code> 或 <code>@contextmanager</code> + yield
多资源	<code>with A() as a, B() as b:</code> 或 <code>ExitStack</code>
泄漏形态	全局容器、无界缓存、闭包捕获、注册不注销
内存排查	<code>tracemalloc</code> 快照 + <code>compare_to</code>
拼接	<code>"\n".join(parts)</code> 优于 <code>+=</code>
小对象	元组/数据类优于 dict;大文件流式处理

最小必会清单

- 能说出引用计数归零、GC、全局引用三种释放时机
- 能解释 `sys.getrefcount` 为什么多 1
- 能构造循环引用并用 `gc.collect` 回收,且不依赖 `__del__`
- 能说出 `__del__` 三宗罪与正确替代
- 能写出 `weakref` 打破循环、`WeakValueDictionary` 做有限缓存
- 能写出自定义上下文管理器(类版与 `@contextmanager` 版)
- 能说出 `with` 与 `try/finally` 的等价关系
- 能说出 GC 语言内存泄漏的四种形态
- 能用 `tracemalloc` 定位内存增长行
- 能写出 `join` 拼接与元组打包的优化写法

自测题

Q 1. `x = [1,2,3]; y = x; del x` 之后,列表对象被释放了吗?为什么?

✅ 参考答案

没有。 `del x` 只删除名字 `x`, 对象仍被 `y` 引用, 计数为 1。只有 `del y` 后计数归零才释放。判断依据是引用计数, 不是“名字是否还存在”。

Q 2. 循环引用为什么会让引用计数失效? Python 靠什么机制解决?

✅ 参考答案

$A \rightarrow B$ 、 $B \rightarrow A$ 时, 两者计数都 ≥ 1 , 永不归零, 即使外部无人引用。解决机制是分代 GC 的标记-清除: 从根(全局、栈、寄存器)出发标记可达对象, 不可达的环被回收。注意只有容器类对象参与 GC。

Q 3. 为什么“用 `__del__` 释放文件句柄”是反模式? 正确做法是什么?

✅ 参考答案

`__del__` 时机不可预测(等 GC、解释器退出不保证), 且循环引用里可能被推迟甚至跳过。正确做法: 实现 `close()` 方法 + 上下文管理器(`__enter__` / `__exit__`), 调用方用 `with` 保证退出时释放。

Q 4. 用 `@contextmanager` 写一个“打印 `with` 块前后”的上下文管理器, 支持 `as` 子句拿到开始时间。

✅ 参考答案

```
import time
from contextlib import contextmanager

@contextmanager
def log_block(name):
    print(f"[{name}] 开始")
    start = time.perf_counter()
    try:
        yield start          # as 子句拿到 start
    finally:
        cost = time.perf_counter() - start
        print(f"[{name}] 结束, 耗时 {cost * 1000:.2f} ms")

with log_block("任务") as t0:
    print(f" 已用 {time.perf_counter() - t0:.3f} s")
```

Q 5. 全局缓存字典为什么可能“泄漏”内存? 给出两种修复。

✅ 参考答案

缓存里的对象被永久强引用, 永不回收。修复: ① `weakref.WeakValueDictionary`, 对象被别处释放后自动出缓存; ② 带上限的缓存(`functools.lru_cache(maxsize=N)` 或手写 LRU)。核心原则: 缓存必须有限。

Q 6. `gc.disable()` 之后会发生什么? 什么情况下它还安全?

✅ 参考答案

循环引用不再被自动回收,长期运行的进程内存只增不减。仅当:①程序无循环引用(全部对象由引用计数管理);②配合显式 `gc.collect()` 定期清理。生产服务禁用 GC 前必须做内存压测,绝大多数情况不该碰这个开关。

章末实验题:内存行为观察与泄漏修复

📄 实验 5:内存行为观察实验(泄漏复现与修复)

实验目标:复现一个"日志服务"的内存泄漏(全局日志列表 + 循环引用 + 无界缓存),用引用计数观察、tracemalloc 定位,最终修复并用上下文管理器管理资源。

实验要求:

- 任务 1:写 `observe_refcount()` :创建对象,观察赋值/入列表/入字典/删除四种操作后的 `sys.getrefcount` 变化(知识点:引用计数)
- 任务 2:定义 `LogEntry` 类(含 payload 大字符串、next/prev 双链引用),构造 1000 个且相互成环,删除外部引用后观察 `len(gc.get_objects())` 与 `gc.collect()` 的回收数(知识点:循环引用、gc 模块)
- 任务 3:给 `LogEntry` 加 `__del__` 打印回收,观察"先构造环再回收"时 `__del__` 的调用时机,体会其不可预测性(知识点: `__del__` 的坑)
- 任务 4:用 `weakref.WeakValueDictionary` 实现日志缓存,对比强引用字典在"对象被删后"的行为差异(知识点:weakref)
- 任务 5:用 `@contextmanager` 实现 `memory_watch(label)` :进入时打印 tracemalloc 当前快照大小,退出时打印增量,用它测量任务 2 的泄漏量(知识点:上下文管理器、tracemalloc)
- 任务 6:综合修复:日志列表改用 `collections.deque(maxlen=200)` ,缓存改 `WeakValueDictionary` ,全部文件操作放 with 里——修复后再跑 `memory_watch` ,确认增量收敛(知识点:生产内存纪律)

实验环境:Python 3.10+,单文件 `memlab.py` 。

第一步 写 `observe_refcount`,把计数变化打印成表格,确认自己的直觉

第二步 构造环时用一个函数 `make_chain(n)` 返回所有节点,外部 `del nodes` 后观察——注意先把 `gc.disable()` 再 `enable` 以便精确观察

第三步 `memory_watch` 用 `contextmanager` 包裹 `tracemalloc.start/stop` 或 `take_snapshot` 对比

第四步 后把修复前后的峰值打印出来,形成对比结论

✅ 参考答案要点(代码分两段)

第一段:引用计数观察与循环引用。设计要点:用 `get_total_size()` 统一报告内存,方便前后对比。

```

# memlab.py — 内存行为观察与泄漏修复(覆盖第5章全部知识点)
import gc
import sys
import time
import weakref
from collections import deque
from contextlib import contextmanager
import tracemalloc

class LogEntry:
    """日志条目:带大 payload 与双向环引用"""

    def __init__(self, i: int):
        self.i = i
        self.payload = f"log-content-{i}" + "x" * 2000
        self.next = None
        self.prev = None

    def __del__(self):
        print(f"[__del__] 条目 {self.i} 被回收")

def observe_refcount() -> None:
    """观察四种操作对引用计数的影响"""
    x = [1, 2, 3]
    print(f"创建后:      {sys.getrefcount(x) - 1}")
    y = x
    print(f"y = x 后:      {sys.getrefcount(x) - 1}")
    lst = [x]
    print(f"入列表后:      {sys.getrefcount(x) - 1}")
    d = {"k": x}
    print(f"入字典后:      {sys.getrefcount(x) - 1}")
    del y, lst, d
    print(f"删除后:      {sys.getrefcount(x) - 1}")
    del x

def make_cycle(n: int) -> list:
    """构造 n 个互相成环的 LogEntry, 返回外部引用列表"""
    nodes = [LogEntry(i) for i in range(n)]
    for i in range(n):
        nodes[i].next = nodes[(i + 1) % n]
        nodes[i].prev = nodes[(i - 1) % n]
    return nodes

@contextmanager
def memory_watch(label: str):
    """进入时记录快照,退出时打印增量(上下文管理器实战)"""
    before = tracemalloc.take_snapshot().statistics("lineno")
    start = sum(s.size for s in before)
    try:
        yield
    finally:

```

```
after = tracemalloc.take_snapshot().statistics("lineno")
end = sum(s.size for s in after)
print(f"[{label}] 内存增量: {(end - start) / 1024:.1f} KiB")
```

第二段:泄漏场景与修复。修复前:强引用缓存 + 无界日志;修复后:弱引用缓存 + 有界队列。

```

def leaky_service(rounds: int) -> None:
    """修复前:全局日志列表只增不减 + 强引用缓存"""
    log_store = []
    cache = {}
    for r in range(rounds):
        nodes = make_cycle(50)          # 环:等 GC
        log_store.append(len(nodes))     # 列表只增不减
        cache[f"key-{r}"] = nodes[0]    # 强引用缓存
    with memory_watch("泄漏版"):
        del nodes, cache, log_store
    gc.collect()

def fixed_service(rounds: int) -> None:
    """修复后:deque 有界日志 + 弱引用缓存 + 显式清环"""
    log_store = deque(maxlen=200)       # 有界:永不膨胀
    cache = weakref.WeakValueDictionary() # 弱引用:自动淘汰
    for r in range(rounds):
        nodes = make_cycle(50)
        log_store.append(len(nodes))
        for node in nodes:              # 断开环,立即回收
            node.next = node.prev = None
        cache[f"key-{r}"] = nodes[0]
    with memory_watch("修复版"):
        del nodes, cache, log_store
    gc.collect()

def main() -> None:
    tracemalloc.start()                 # 追踪开始
    print("== 引用计数观察 ==")
    observe_refcount()

    print("\n== 循环引用回收(GC 关闭时) ==")
    gc.disable()
    before = len(gc.get_objects())
    nodes = make_cycle(200)
    del nodes                           # 外部引用消失,环仍在
    gc.collect()                        # 手动回收(有 __del__ 的会进 garbage)
    print(f"回收了 {before - len(gc.get_objects())} 个对象")
    print(f"gc.garbage 中的对象数: {len(gc.garbage)}")
    gc.garbage.clear()
    gc.enable()

    print("\n== 泄漏 vs 修复 ==")
    leaky_service(20)
    fixed_service(20)

if __name__ == "__main__":
    main()

```

设计说明:① 观察实验放在函数里,保证局部变量作用域可控;② 循环引用实验先 `gc.disable()` 再手动 `collect`,让学生看到"外部引用消失≠立即回收";③ 有 `__del__` 的环回收后会残留到 `gc.garbage`,这正是"__del__ 让 GC 失

效"的经典表现;④ 修复版通过断开环引用让引用计数立即生效,配合有界队列与弱引用缓存,内存增量收敛;⑤ memory_watch 上下文管理器同时复习了第5章 with 与第4章闭包/装饰器思想。

下一章预告:第6章 面向对象——类与 self、_ 私有约定、继承与 MRO、鸭子类型多态、运算符重载、@classmethod/@staticmethod、组合优于继承。